

TAKT: A Formal Framework for Minimal Decision-Preserving Runtime Governance

Valentin Lineiro
TAKT Research Initiative
DOI: 10.5281/zenodo.21638014

2026-07-28

Abstract

Software systems operating under abstracted or compressed state representations frequently risk decision instability when critical informational context is discarded. This paper introduces TAKT (Theory of Adequate Knowledge for Decisions), a formal and empirical framework for characterizing minimal governance kernels of execution runtimes. Within the formal ST-016 runtime model, we develop a Lean 4 algebraic framework defining runtime capability necessity, decision preservation, sufficiency, and irreducibility, and demonstrate—on finite witness instances within the model—that three capability classes — `ContractSoundness` (C_{contract}), `UncertaintyBound` ($C_{\text{uncertainty}}$), and `TemporalConsistency` (C_{temporal}) — each have formalized witness scenarios where their removal violates decision preservation. We validate these witnesses on a reference TypeScript runtime via component ablation (EXP-004), connected to the formal model via a *formal witness validation schema*. We present an audited zero-contact replication package across six independent audit rounds, backed by an immutable Zenodo archive (DOI: 10.5281/zenodo.21638014).

Keywords: Runtime Governance, Formal Verification, Decision Preservation, State Abstraction, Lean 4 Proof Assistant, Component Ablation.

1 Introduction

As autonomous software agents and complex reactive systems scale, they continuously compress high-dimensional environmental context into abstract state representations $R = \rho(S)$. However, ungoverned state abstraction introduces decision divergence: an optimal policy $\pi_S^* : \mathcal{S} \rightarrow \mathcal{D}$ computed on full observable state S may fail when the governed runtime’s policy $\pi_M : \mathcal{R} \rightarrow \mathcal{D}$ is evaluated on compressed representation R . This phenomenon has been studied across formal verification [2, 4], decision theory [1], and

control under partial observability [3], yet a formal algebraic framework for characterizing the *minimal governance capability kernel* required for decision preservation has not been established.

This work addresses two fundamental research questions:

1. *What information must be preserved to guarantee decision equivalence?* (ST-015)
2. *What minimal runtime capability composition suffices and is irreducible for decision preservation, within a formal runtime model?* (ST-016)

1.1 Main Contributions

This paper makes four primary contributions:

1. **Algebraic Framework for Runtime Governance:** We formalize capability necessity, decision preservation, runtime sufficiency, and irreducibility under state abstraction in Lean 4.
2. **Formalized Witness Scenarios** (*within the formal ST-016 model*): We construct Lean 4 certified witness scenarios demonstrating that removing any of $\{C_{\text{contract}}, C_{\text{uncertainty}}, C_{\text{temporal}}\}$ from the formal model produces decision preservation violations for specific representations.
3. **Formal Witness Validation Schema:** We introduce a structured schema connecting empirical ablation observations (EXP-004) to the formal model: witness artifacts are manually constructed from TypeScript runtime traces and formally verified against the Lean model via `WitnessConsistentWithRuntime`.
4. **Audited Zero-Contact Replication Package:** We publish an immutable, versioned research package [5] with multi-OS CI and an external audit trail of 6 independent dry runs.

2 Related Work & Positioning

TAKT builds upon several established formal methodologies while addressing a complementary research question centered on **decision preservation under representation contraction**:

- **Abstract Interpretation [2]:** Focuses on sound static over-approximation of program semantics via Galois connections. TAKT focuses on dynamic policy decision invariance ($\pi^*(R) = \pi^*(S)$) rather than total semantic property preservation.

- **Bisimulation & Process Refinement** [6, 7]: Enforces strict state-by-state behavioral equivalence between transition systems. TAKT permits aggressive internal state contraction provided temporal prefix monitoring (C_{temporal}) maintains policy consistency across trajectory prefixes.
- **Runtime Verification** [4]: Evaluates execution traces dynamically against temporal logic formulas (LTL/ERE). TAKT formalizes and machine-checks the *necessity of the governance runtime kernel itself*, bridging empirical trace witnesses to Lean 4 necessity theorems.
- **Statistical Decision Theory** [1]: Establishes the foundational notion of observational sufficiency for decision problems. TAKT operationalizes this into an irreducible 3-capability runtime kernel $\mathcal{K}_D$.
- **Planning Under Partial Observability** [3]: Provides optimal control frameworks under belief-state uncertainty (POMDP). TAKT provides formal necessity certification for governance components rather than belief update algorithms.

Unlike these paradigms, ST-016 directly formalizes the *minimal governance capability kernel* required for decision preservation, connecting formal proofs to independently audited empirical witnesses [5].

3 Theoretical Foundations & Runtime Necessity Model

3.1 Decision-Preserving Sufficiency (ST-015)

Let $\pi_S^* : \mathcal{S} \rightarrow \mathcal{D}$ denote the optimal decision policy over full states, and $\pi_M : \mathcal{R} \rightarrow \mathcal{D}$ the policy executed by governed runtime M . A representation $R = \rho(S)$ is *decision-sufficient* when the runtime produces the same decision as the optimal full-state policy:

$$\forall S \in \mathcal{S} : \quad \pi_M(\rho(S)) = \pi_S^*(S)$$

This condition says: *no matter which full state S the system is in, if the runtime sees its compressed representation $\rho(S)$, it makes the same decision that the optimal full-state policy would make on S .*

3.2 From Informational Sufficiency to Runtime Necessity (ST-016)

ST-015 answers: *what information must not be lost in compression?* ST-016 answers: *what runtime mechanisms are required to preserve that information dynamically during execution?*

The connection is direct: if removing a runtime capability C causes the runtime to discard information that π_S^* requires to distinguish two states, then $\pi_M(\rho(S)) \neq \pi_S^*(S)$ for some S , and decision preservation breaks. ST-016 formalizes exactly which capability classes produce this breakage in the reference governance model.

3.3 Runtime Necessity Formalization (ST-016)

We formalize a TAKT Governed Runtime as a tuple $M = (\mathcal{C}, \pi_M)$, where $\mathcal{C}$ is the runtime’s capability set and $\pi_M : \mathcal{R} \rightarrow \mathcal{D}$ is its decision policy. The *minimal governance kernel* is defined as:

$$\mathcal{K}_D = \{C_{\text{contract}}, C_{\text{uncertainty}}, C_{\text{temporal}}\}$$

A runtime M is *minimally governed* when $\mathcal{C} = \mathcal{K}_D$.

Three definitions of decision preservation appear at different layers of abstraction, all specializations of the same core condition:

- **Full-state (ST-015):** $\pi_M(\rho(S)) = \pi_S^*(S)$ for all $S \in \mathcal{S}$.
- **Runtime sufficiency:** $\text{Sufficient}(M, \pi_S^*) \equiv \forall R \in \mathcal{R}, \pi_M(R) = \pi_S^*(\rho^{-1}(R))$.
- **Cross-runtime:** $\text{PreservesDecision}(M_1, M_2) \equiv \forall R, \pi_{M_1}(R) = \pi_{M_2}(R)$.

Definition (Capability Necessity): A capability $C \in \mathcal{C}$ is *locally necessary* for runtime M if removing C alters policy decisions on at least one representation:

$$\exists R \in \mathcal{R} : \pi_M(R) \neq \pi_{M \setminus \{C\}}(R)$$

Axiom0 (Kernel Coincidence): The structural sufficiency module postulates:

$$\text{kernel}_D(x, y) \leftrightarrow K_D(x, y)$$

Informally: two states x and y are indistinguishable for the decision task (i.e., any optimal policy treats them identically) if and only if they preserve all task-relevant capability relations. This is a modelling assumption characterizing the class of decision problems studied; it is listed in the Non-Claims of Section 7.

Example 1 (Temporal Prefix Necessity): Consider a runtime where C_{temporal} is ablated. Two trajectory-induced representations R^1 and R^2 share an identical terminal observation yet arise from trajectories $\tau_1 = (r_0, r_1, r_2)$ (safe) and $\tau_2 = (r'_0, r'_1, r_2)$ (unsafe). Here π_S^* is defined over full states that encode trajectory history; $R^1 \neq R^2$ because their history components differ. A runtime inspecting only the terminal state r_2 produces: $\pi_{M \setminus \{C_{\text{temporal}}\}}(R^1) = \pi_{M \setminus \{C_{\text{temporal}}\}}(R^2)$, while π_M correctly distinguishes them: $\pi_M(R^1) \neq \pi_M(R^2)$, violating $\pi_M(\rho(S)) = \pi_S^*(S)$.

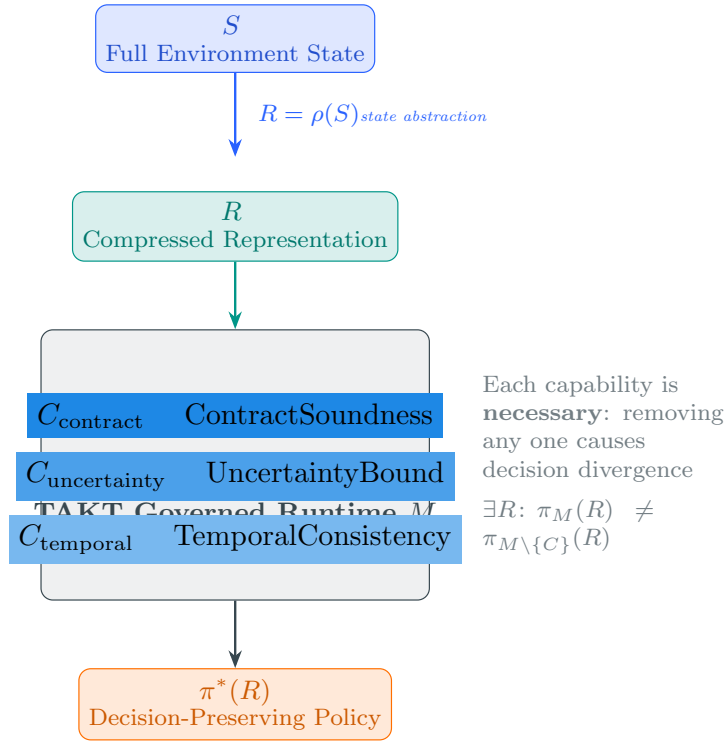


Figure 1: TAKT runtime M enforces $\pi_M(\rho(S)) = \pi_S^*(S)$ under abstraction.

4 Machine-Certified Lean 4 Formalization

The Lean 4 development in `takt-formal` provides the algebraic framework within which the ST-016 runtime model is defined and analyzed.

4.1 Core Definitions (`RuntimeSufficiency.lean`)

The key definitions, verbatim from the Lean source:

```
inductive RuntimeCapability
  | contract | uncertainty | temporal

structure Runtime (S : Type) (D : Type) where
  capabilities : Set RuntimeCapability
  policy       : S -> D

def NecessaryCapability (C : RuntimeCapability) (M : Runtime S D) : Prop :=
  Exists r, M.policy r != (removeCapability M C).policy r

def Sufficient (M : Runtime S D) (pi_star : S -> D) : Prop :=
  forall r, M.policy r = pi_star r

def MinimalRuntime (M : Runtime S D) (pi_star : S -> D) : Prop :=
  Sufficient M pi_star /\ Irreducible M
```

As a definitional consequence, any `MinimalRuntime` has all its capabilities necessary by construction (`Irreducible` is the second conjunct). The non-trivial lemma `necessary_implies_non_preservation` proves that a necessary capability, when removed, strictly breaks `PreservesDecision`.

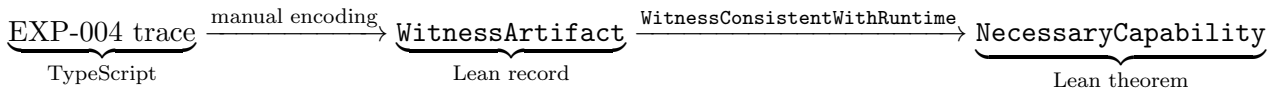
4.2 Formal Witness Validation Schema (`RuntimeWitness.lean`)

A `WitnessArtifact` is a record encoding a single observed policy divergence under capability ablation. Table 1 shows a concrete witness for C_{temporal} drawn from EXP-004.

Table 1: Concrete `WitnessArtifact` for C_{temporal} (EXP-004, ablation 3)

Field	Value
<code>cap</code>	<code>RuntimeCapability.temporal</code>
<code>representation</code>	R2 (unsafe prefix, same terminal state)
<code>fullDecision</code>	INTERVENE
<code>reducedDecision</code>	MONITOR
<code>divergence</code>	<code>true</code> ($\pi_M(R^2) \neq \pi_{M \setminus \{C_{\text{temporal}}\}}(R^2)$)

The construction and verification pipeline is:



The first arrow is manual: a human observes the TypeScript divergence and encodes it into a `WitnessArtifact` record. The second arrow is machine-checked: Lean verifies that if the record is consistent with the formal runtime model, then the corresponding capability is formally necessary within that model.

4.3 Formal to Runtime Mapping

Table 2: Formal model to reference runtime mapping

Formal (Lean)	concept	Runtime counterpart (TypeScript)
<code>RuntimeCapability.contract</code>	Contract	Contract validator layer
<code>RuntimeCapability.uncertainty</code>	Debt	Debt margin evaluator
<code>RuntimeCapability.temporal</code>	History	History / prefix monitor
<code>removeCapability M C</code>		Runtime with component disabled
<code>WitnessArtifact</code>		EXP-004 recorded divergence record
<code>WitnessConsistentWithRuntime</code>		Formal consistency check predicate

Both modules compile with zero errors and zero `sorrys` [5].

5 Empirical Validation & Component Ablation (EXP-004)

5.1 Experimental Protocol

EXP-004 [5] ablates one capability at a time from the reference TypeScript runtime and records whether the runtime’s decision diverges from the full-capability baseline on a designated trigger scenario. Each scenario is designed so that the ablated capability is the deciding factor.

Each ablation divergence is manually encoded into a `WitnessArtifact` record, encoding the experimentally observed divergence as a formally checkable witness verified against the formal model via `WitnessConsistentWithRuntime`. When consistency holds, `NecessaryCapability` is established within the Lean model.

5.2 Evidence Traceability Matrix

Table 4 maps theoretical claims to primary evidence, with explicit scope annotations distinguishing formal model results from TypeScript-level results.

Table 3: EXP-004 Ablation Protocol and Results

Ablation	Trigger scenario	sce-	Full policy	Ablated policy	Result
C_{contract}	Contract-violating state		STOP	EXECUTE	Diverge
$C_{\text{uncertainty}}$	Margin boundary ($M_D \approx 0$)		REFINE	EXECUTE	Diverge
C_{temporal}	History-dependent prefix		INTERVENE	MONITOR	Diverge
None (base-line)	(base-line)	All trigger scenarios	Correct	N/A	Stable

Table 4: ST-016 Evidence Traceability Matrix

Claim	Primary Evidence	Scope
<i>Formal model (Lean 4)</i>		
Structural sufficiency	StructuralSufficiency	Lean model
Framework definitions	RuntimeSufficiency	Lean model
Witness validation	RuntimeWitness	Lean model
<i>Witness scenarios (finite instances)</i>		
C_{contract} witness	contract.ablation	3 scenarios
$C_{\text{uncertainty}}$ witness	uncertainty.ablation	3 scenarios
C_{temporal} witness	temporal.ablation	3 scenarios
<i>Runtime engineering (TypeScript)</i>		
Integration stability	Vitest (283/283, 76 files)	TypeScript
Reproducibility [5]	Replication kit	6 Runs

6 Discussion & Architectural Implications

The ST-016 results yield three actionable design principles for runtime system architects. Each principle is grounded in a specific formal or empirical result from this work.

1. **Safety Contract Decoupling:** Safety verification (C_{contract}) cannot be subsumed by statistical policy estimation alone. *Grounded in:* EXP-004 ablation of C_{contract} produced EXECUTE on a contract-violating state where the full runtime correctly returned STOP. The formal witness artifact was verified consistent with the Lean model.
2. **Dynamic Boundary Monitoring:** Operating near ambiguity boundaries requires explicit uncertainty estimation ($C_{\text{uncertainty}}$) to prevent premature execution. *Grounded in:* EXP-004 ablation of $C_{\text{uncertainty}}$ at decision margin $M_D \approx 0$ produced EXECUTE where the full runtime returned REFINE, with the divergence formally certified via `RuntimeWitness`.
3. **Trajectory History Dependence:** Stateless evaluation fails under history-dependent processes, necessitating explicit trajectory prefix tracking via C_{temporal} . *Grounded in:* EXP-004 ablation of C_{temporal} produced MONITOR on a history-dependent prefix where the full runtime returned INTERVENE (see also Example 1, Section 3).

7 Scope Boundaries, Non-Claims & Future Work (ST-017)

7.1 Non-Claims & Scope Boundaries

1. **Model Scope:** ST-016 establishes an algebraic framework and witness-based demonstration that C_{contract} , $C_{\text{uncertainty}}$, and C_{temporal} are necessary in formalized finite-instance scenarios. It does *not* claim universal necessity for all possible governed runtimes, continuous control systems, or runtime models other than the ST-016 formal model.
2. **Capability Semantics:** The formal Lean model treats C_{contract} , $C_{\text{uncertainty}}$, and C_{temporal} as abstract labels. The semantic connection to contract soundness, uncertainty bounding, and temporal consistency is established through finite-instance examples. General semantic grounding is reserved for future work.
3. **Axiom0:** The structural sufficiency module postulates $\text{kernel}_D(x, y) \leftrightarrow K_D(x, y)$. This is a modelling assumption characterizing the class of decision problems studied, not a derived theorem.

4. **Witness Bridge Automation:** The formal witness validation schema requires manual construction of `WitnessArtifact` records from observed TypeScript traces. No automatic extraction mechanism from runtime executions to Lean proofs exists in ST-016.
5. **Transportability (ST-017 Boundary):** Transporting certified witness artifacts across heterogeneous runtimes ($M_1 \sim M_2$) is reserved for **ST-017**.

7.2 Open Research Direction: ST016_Conjecture

The following claim is **not proven** in this work and constitutes the central open research direction for ST-017:

ST016 Conjecture: For any runtime M with $\mathcal{C} = \{C_{\text{contract}}, C_{\text{uncertainty}}, C_{\text{temporal}}\}$ and any optimal policy π^* , if M is decision-preserving then M is minimal (i.e., no capability can be removed while preserving π^*).

Proving this conjecture would require establishing formal semantic axioms for each capability class and demonstrating irreducibility in a model-independent way, which is the core research question of ST-017.

7.3 Conclusion

ST-016 establishes an algebraic Lean 4 framework for reasoning about minimal decision-preserving runtime governance, backed by formalized witness scenarios and an audited replication package. The central open conjecture and semantic grounding of capability classes provide the foundation for the ST-017 research line on transportability and generalization.

References

- [1] David Blackwell. Comparison of experiments. In *Proceedings of the Second Berkeley Symposium on Mathematical Statistics and Probability*, pages 93–102. University of California Press, 1951.
- [2] Patrick Cousot and Radhia Cousot. Abstract interpretation: a unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *Proceedings of the 4th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages (POPL)*, pages 238–252. ACM, 1977.
- [3] Leslie Pack Kaelbling, Michael L. Littman, and Anthony R. Cassandra. Planning and acting in partially observable stochastic domains. *Artificial Intelligence*, 101(1-2):99–134, 1998.

- [4] Martin Leucker and Christian Schallhart. A brief account of runtime verification. *Journal of Logic and Algebraic Programming*, 78(5):293–303, 2009.
- [5] Valentin Lineiro. TAKT ST-016: Minimal decision-preserving runtime governance standard. Zenodo, 2026. Version 1.0.0, Zenodo DOI: 10.5281/zenodo.21638014.
- [6] Robin Milner. *Communication and Concurrency*. Prentice Hall, 1989.
- [7] David Park. Concurrency and automata on infinite sequences. In *Theoretical Computer Science (5th GI-Conference)*, pages 167–183. Springer, 1981.